

Практическое задание по лекции 10

Прогон программы, демонстрирующей передачу информации от одного процесса к другому через разделяемую память

First.c

```
#include <windows.h>
#include <stdio.h>
void main(void){
HANDLE hMapFile;
LPVOID lpMapAddress;
HANDLE hFile;
char * String;
hFile = CreateFile("MyFile.txt", // имя файла
GENERIC_READ | GENERIC_WRITE, // файл для чтения и записи
FILE_SHARE_READ| FILE_SHARE_WRITE, // режим совместного доступа
NULL, // защита по умолчанию
OPEN_EXISTING, // файл должен существовать
FILE_ATTRIBUTE_NORMAL, // атрибуты файла
NULL); // файл атрибутов
if (hFile == INVALID_HANDLE_VALUE) printf("Could not open file\n");
hMapFile = CreateFileMapping(hFile, // описатель отображаемого файла
NULL, // атрибуты защиты по умолчанию
PAGE_READWRITE, // режим доступа
0, // старшее двойное слово размера буфера
0, // младшее двойное слово размера буфера
"MyFileObject"); // имя объекта
if (hMapFile == NULL) printf("Could not create file-mapping object.\n");
lpMapAddress = MapViewOfFile(hMapFile, // описатель отображаемого файла
FILE_MAP_ALL_ACCESS, // режимы доступа
0, 0, // отображение файла с начала
0); // отображение целого файла
if (lpMapAddress == NULL) printf("Could not map view of file.\n");
String = (char *)lpMapAddress;
sprintf(String, "Hello, world");
getchar();
}
```

Second.c

```
#include <windows.h>
#include <stdio.h>
void main(void){
HANDLE hMapFile;
LPVOID lpMapAddress;
HANDLE hFile;
char * String;
hFile = CreateFile("MyFile.txt", // имя файла
GENERIC_READ | GENERIC_WRITE, // файл для чтения и записи
FILE_SHARE_READ| FILE_SHARE_WRITE, // режим совместного доступа
```

```

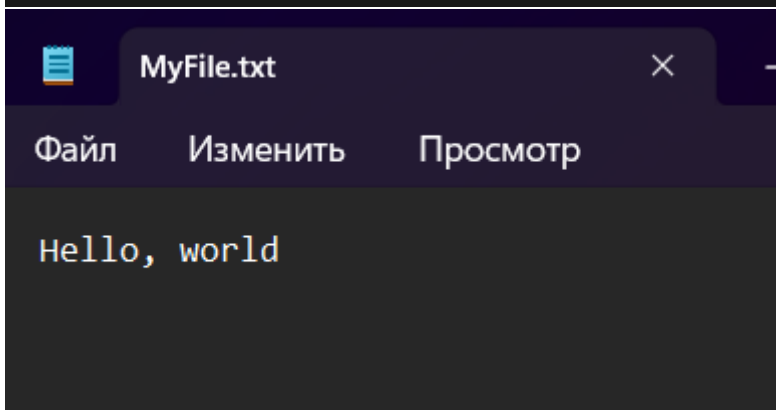
NULL, // защита по умолчанию
OPEN_EXISTING, // файл должен существовать
FILE_ATTRIBUTE_NORMAL, // атрибуты файла
NULL); // файл атрибутов
if (hFile == INVALID_HANDLE_VALUE)
{
printf("Could not open file\n"); // process error
}
hMapFile = OpenFileMapping(FILE_MAP_ALL_ACCESS, // разрешение чтения-записи
FALSE, // описатель не наследуется
"MyFileObject"); // имя объекта проецируемого файла
if (hMapFile == NULL) printf("Could not open Filemapping\n");
lpMapAddress = MapViewOfFile(hMapFile, // описатель отображаемого файла
FILE_MAP_ALL_ACCESS, // режимы доступа
0, 0, // отображение файла с начала
0); // отображение целого файла
if (lpMapAddress == NULL) printf("Could not map view of file.\n");
String = (char *)lpMapAddress;
printf("%s\n", String);
getchar();
}

```

```

PS C:\Users\diamo\OneDrive\Документы\2 курс\ос\лаб 10>
Hello, world

```



Прогон программы, иллюстрирующей увеличение рабочего набора процесса

```

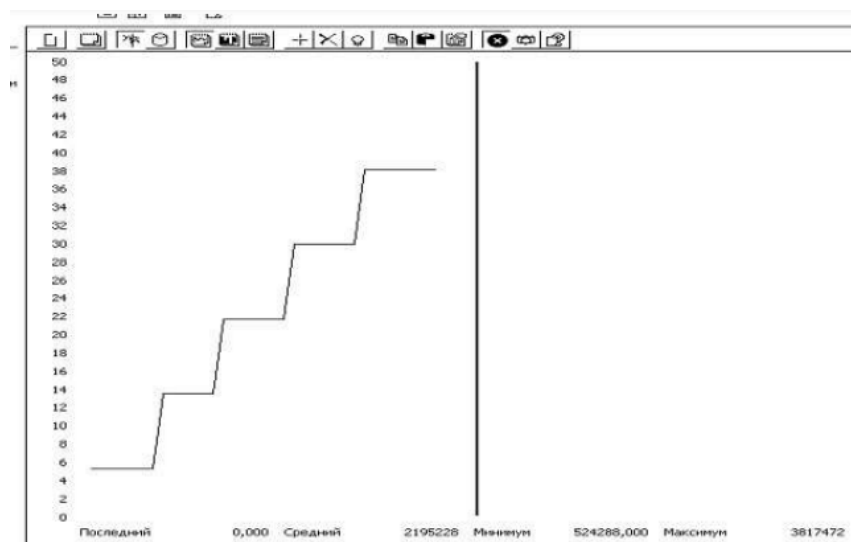
#include <windows.h>
#include <stdio.h>
void main(void)
{
PVOID pMem = NULL;
int nPageSize = 4096;
int nPages = 200;
long SizeCommit = 0;
int i;
char * Ptr;
SizeCommit = nPages * nPageSize;

```

```

getchar();
pMem = VirtualAlloc(0, SizeCommit, MEM_RESERVE| MEM_COMMIT,
PAGE_READWRITE);
if(pMem == NULL) printf("VirtualAlloc Error\n");
Ptr = (char *)pMem;
for(i=0; i<nPages; i++) Ptr[i*nPageSize] = '0';
getchar();
pMem = VirtualAlloc(0, SizeCommit, MEM_RESERVE| MEM_COMMIT,
PAGE_READWRITE);
if(pMem == NULL) printf("VirtualAlloc Error\n");
Ptr = (char *)pMem;
for(i=0; i<nPages; i++) Ptr[i*nPageSize] = '0';
getchar();
pMem = VirtualAlloc(0, SizeCommit, MEM_RESERVE| MEM_COMMIT,
PAGE_READWRITE);
if(pMem == NULL) printf("VirtualAlloc Error\n");
Ptr = (char *)pMem;
for(i=0; i<nPages; i++) Ptr[i*nPageSize] = '0';
getchar();
pMem = VirtualAlloc(0, SizeCommit, MEM_RESERVE| MEM_COMMIT,
PAGE_READWRITE);
if(pMem == NULL) printf("VirtualAlloc Error\n");
Ptr = (char *)pMem;
for(i=0; i<nPages; i++) Ptr[i*nPageSize] = '0';
getchar();
}

```



Прогон программы, демонстрирующей блокировку страниц в памяти

```

#include <windows.h>
#include <stdio.h>
void main(void)
{
PVOID pMem = NULL;

```

```

int nPageSize = 4096;
int nPages = 400;
int nPageLock = 100;
long SizeCommit = 0;
int i;
char * Ptr;
int nMinPages = 200, nMaxPages = 500;
long dwMinimumWorkingSetSize = 0, dwMaximumWorkingSetSize = 0;
HANDLE hProcess;
hProcess = GetCurrentProcess();
dwMinimumWorkingSetSize = nMinPages * nPageSize;
dwMaximumWorkingSetSize = nMaxPages * nPageSize;
i = SetProcessWorkingSetSize(hProcess, dwMinimumWorkingSetSize,
dwMaximumWorkingSetSize);
if(i==0) printf("SetProcessWorkingSetSize Error\n");
SizeCommit = nPages * nPageSize;
pMem = VirtualAlloc(0, SizeCommit, MEM_RESERVE | MEM_COMMIT,
PAGE_READWRITE);
if(pMem == NULL) printf("VirtualAlloc Error\n");
Ptr = (char *)pMem;
for(i=0; i<nPages; i++) Ptr[i*nPageSize] = '0';
i = VirtualLock(pMem, nPageLock * nPageSize);
if(i==0) printf("VirtualLock Error\n");
for(i=0; i<nPages; i++) Ptr[i*nPageSize] = '0';
VirtualUnlock(pMem, nPageLock * nPageSize);
VirtualFree(pMem, 0, MEM_RELEASE);
}

```

Прогон программы, иллюстрирующей отложенное выделение памяти.

```

#include <windows.h>
#include <stdio.h>
void main(void)
{
HANDLE hMapFile;
LPVOID lpMapAddress;
HANDLE hFile;
char * String;
hFile = CreateFile("MyFile.txt", GENERIC_READ | GENERIC_WRITE,
FILE_SHARE_READ | FILE_SHARE_WRITE, NULL, OPEN_ALWAYS,
FILE_ATTRIBUTE_NORMAL, NULL);
if (hFile == INVALID_HANDLE_VALUE) printf("Could not open file\n");
hMapFile = CreateFileMapping(hFile, NULL,
PAGE_WRITECOPY, // копирование при записи
0, 0, "MyFileObject");
if (hMapFile == NULL) printf("Could not create file-mapping object.\n");
lpMapAddress = MapViewOfFile(hMapFile,
FILE_MAP_COPY, // копирование при записи
0, 0, 0);

```

```
if (lpMapAddress == NULL) printf("Could not map view of file.\n");
String = (char *)lpMapAddress;
getchar();
sprintf(String, "Hello, world");
printf("%s\n", String);
if (!UnmapViewOfFile(lpMapAddress)) printf("Could not unmap view of file.\n");
}
```

Вывод:

```
PS C:\Users\diamo\OneDrive\Документы\2 курс\ос\лаб 10> & 'c:\U
auncher.exe' '--stdin=Microsoft-MIEngine-In-y1fiuumx.2iw' '--st
=Microsoft-MIEngine-Pid-2khmnyea.050' '--dbgExe=C:\msys64\mingw
Hello, world
```